Consuming Senders from Coroutine-Native Code

Document Number:	D4055R0
Date:	2026-03-13
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com Steve Gerbino steve@gerbino.co

Table of Contents

Abstract

A coroutine type satisfying `IoAwaitable` (P4003R0[3]) consumes `std::execution` senders with zero allocations, correct stop token propagation, and automatic executor dispatch-back. The bridge is one class template. The complete implementation is in Appendix A. This paper is informational and proposes no action.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The authors developed Capy[4] and Corosio[6]. Capy[4] is a coroutine primitives library. It contains no sockets, no timers, and no platform-specific I/O APIs. Networking is in Corosio[6], which is not used in this paper. The bridge depends on Capy[4] and `beman::execution` [5], a community implementation of `std::execution` (P2300R10[1]). Source code links refer to pinned commit 60bf781[4].

---

2. The Bridge

```
capy::task<int> compute(auto sched)
{
    int result = co_await capy::await_sender(
        ex::schedule(sched)
            | ex::then([] {
                std::cout
                    << "  sender running on thread "
                    << std::this_thread::get_id()
                    << "\n";
                return 42 * 42;
            }));

    std::cout
        << "  coroutine resumed on thread "
        << std::this_thread::get_id() << "\n";

    co_return result;
}
```

`await_sender` returns a `sender_awaitable` that satisfies `IoAwaitable` (P4003R0[3]). Any coroutine type whose promise propagates `io_env` through `await_suspend(h, io_env const*)` can use it. `capy::task` is one such type. It is not the only one.

The bridge is one class template. The complete implementation is in Appendix A. It was tested against `beman::execution` [5].

---

3. Demonstration

```
main thread: 32208
  sender running on thread 9560
  coroutine resumed on thread 34356
result: 1764
```

The sender executed on thread 9560, the `beman::execution::run_loop` thread. The coroutine resumed on thread 34356, the Capy thread pool. The value 1764 crossed the bridge. The bridge allocated zero bytes.

---

4. What the Bridge Does

The bridge consumes any `std::execution` sender. Scheduler hops work. Stop token propagation works. `set_value`, `set_error`, and `set_stopped` are handled. The operation state is stored inline with zero allocations. Any sender pipeline - `when_all`, `then`, `let_value`, `on` - works through the bridge.

The bridge does not use `execution::task`.

---

5. What the Bridge Does Not Require

Property	`execution::task`	Bridge
Routine I/O errors become exceptions	Yes	No
Type erasure on connect	Yes	No
`dispatch` adapter for compound results	Yes	No

Property	`execution::task`	Bridge
Zero allocations in the bridge	No	Yes

`std::execution::task` is not necessary to consume senders.

---

6. The Narrowest Abstraction

Capy[4] is a coroutine primitives library. It provides `task`, executors, stop tokens, frame allocators, and buffer sequences. It contains no sockets, no timers, and no platform-specific I/O APIs. Networking is in Corosio[6], a separate library that depends on Capy[4].

The sender bridge depends on Capy[4] and `std::execution`. It does not depend on Corosio[6]. It does not depend on any platform I/O API. A user who needs I/O adds Corosio[6]. A user who needs sender interop adds the bridge. A user who needs neither does not pay for either. The coroutine type does not change between these configurations.

---

7. Acknowledgments

The authors thank Ville Voutilainen and Jens Maurer for reflector discussion on dispatch patterns, Herb Sutter for identifying the need for tutorials and documentation, Mark Hoemmen for insights on `std::linalg` and the layered abstraction model, and Dietmar Kühl for `beman::execution`.

References

1. P2300R10 - “std::execution” (Michał Dominiak et al., 2024). https://wg21.link/p2300r10
2. P3552R3 - “Add a Coroutine Task Type” (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
3. P4003R0 - “IoAwaitable” (Vinnie Falco, 2026). https://wg21.link/p4003r0
4. cppalliance/capy - Coroutine primitives library. Commit 60bf781. https://github.com/cppalliance/capy/tree/60bf781d09024ce0f9cbd1505684249184331692

5. bemanproject/execution - Community implementation of `std::execution`.
   https://github.com/bemanproject/execution

6. cppalliance/corosio - Coroutine-native networking library. https://github.com/cppalliance/corosio

---

Appendix A. Bridge Implementation

```
#include <boost/capy/ex/io_env.hpp>

#include <beman/execution/execution.hpp>

#include <coroutine>
#include <cstring>
#include <exception>
#include <new>
#include <stop_token>
#include <tuple>
#include <type_traits>
#include <variant>

namespace boost::capy {

namespace detail {

struct stopped_t {};

struct bridge_env
{
    std::stop_token st_;

    auto query(
        beman::execution::get_stop_token_t const&)
            const noexcept
    {
        return st_;
    }
};

template<class Sender>
using sender_single_value_t =
    beman::execution::value_types_of_t<
        Sender,
        bridge_env,
        std::tuple,
        std::type_identity_t>;

template<class ValueTuple>
struct bridge_receiver
{
    using receiver_concept =
```

```
        beman::execution::receiver_t;

    std::variant<
        std::monostate,
        ValueTuple,
        std::exception_ptr,
        stopped_t>*         result_;
    std::coroutine_handle<> cont_;
    io_env const*           env_;

    auto get_env() const noexcept -> bridge_env
    {
        return {env_->stop_token};
    }

    template<class... Args>
    void set_value(Args&&... args) && noexcept
    {
        result_->template emplace<1>(
            std::forward<Args>(args)...);
        env_->executor.post(cont_);
    }

    template<class E>
    void set_error(E&& e) && noexcept
    {
        if constexpr (
            std::is_same_v<
                std::decay_t<E>,
                std::exception_ptr>)
            result_->template emplace<2>(
                std::forward<E>(e));
        else
            result_->template emplace<2>(
                std::make_exception_ptr(
                    std::forward<E>(e)));
        env_->executor.post(cont_);
    }

    void set_stopped() && noexcept
    {
        result_->template emplace<3>(
            stopped_t{});
        env_->executor.post(cont_);
    }
};
```

```
} // namespace detail

template<class Sender>
struct sender_awaitable
{
    using value_tuple =
        detail::sender_single_value_t<Sender>;
    using receiver_type =
        detail::bridge_receiver<value_tuple>;
    using op_state_type = decltype(
        beman::execution::connect(
            std::declval<Sender>(),
            std::declval<receiver_type>()));

    Sender sndr_;

    std::variant<
        std::monostate,
        value_tuple,
        std::exception_ptr,
        detail::stopped_t> result_{};

    alignas(op_state_type)
    unsigned char op_buf_[sizeof(op_state_type)];
    bool op_constructed_ = false;

    explicit sender_awaitable(Sender sndr)
        : sndr_(std::move(sndr))
    {
    }

    sender_awaitable(sender_awaitable&& o)
        noexcept(
            std::is_nothrow_move_constructible_v<
                Sender>)
        : sndr_(std::move(o.sndr_))
    {
    }

    sender_awaitable(
        sender_awaitable const&) = delete;
    sender_awaitable& operator=(
        sender_awaitable const&) = delete;
    sender_awaitable& operator=(
        sender_awaitable&&) = delete;

    ~sender_awaitable()
```

```
    {
        if(op_constructed_)
            std::launder(
                reinterpret_cast<op_state_type*>(
                    op_buf_))->~op_state_type();
    }

    bool await_ready() const noexcept
    {
        return false;
    }

    std::coroutine_handle<>
    await_suspend(
        std::coroutine_handle<> h,
        io_env const* env)
    {
        ::new(op_buf_) op_state_type(
            beman::execution::connect(
                std::move(sndr_),
                receiver_type{
                    &result_, h, env}));
        op_constructed_ = true;
        beman::execution::start(
            *std::launder(
                reinterpret_cast<
                    op_state_type*>(
                        op_buf_)));
        return std::noop_coroutine();
    }

    auto await_resume()
    {
        if(result_.index() == 2)
            std::rethrow_exception(
                std::get<2>(result_));
        if(result_.index() == 3)
            throw std::runtime_error(
                "sender completed with "
                "set_stopped");

        if constexpr (
            std::tuple_size_v<
                value_tuple> == 0)
            return;
        else if constexpr (
            std::tuple_size_v<
```

```
                value_tuple> == 1)
            return std::get<0>(
                std::get<1>(
                    std::move(result_)));
        else
            return std::get<1>(
                std::move(result_));
    }
};

template<class Sender>
auto await_sender(Sender&& sndr)
{
    return sender_awaitable<
        std::decay_t<Sender>>(
            std::forward<Sender>(sndr));
}

} // namespace boost::capy
```